

How CDNs Make the Web Fast

From 200ms round-trips to 20ms edge responses. The engineering behind global content delivery.

ENEB352 - Final Research Presentation - Spring 2026

The Internet Wasn't Built for Distance

When a user in Tokyo requests a website hosted in Virginia, the packet travels approximately 11,000 kilometers through fiber optic cables. At two-thirds the speed of light, that is a 70ms one-way trip, 140ms round-trip.

Add TCP three-way handshake (1 RTT, as covered in our TCP lab), TLS negotiation (1-2 RTTs), server processing, and database queries, and you are looking at 300-500ms before the browser receives the first byte.

200ms RTT Cross-continent | 20ms RTT Local edge | 90% Traffic Offloaded

A Stack of Protocols

CDNs orchestrate multiple networking protocols across the OSI stack. At the network and transport layers, we have IP Anycast, BGP for routing, TCP with BBR congestion control, and UDP for QUIC.

At the application layer, DNS resolves names to IPs, HTTP delivers content, and TLS encrypts everything. As we studied in Kurose and Ross Chapter 2, these protocols work together to deliver web content efficiently.

Protocols: DNS, Anycast, BGP, TCP, TLS 1.3, HTTP/1.1, HTTP/2, HTTP/3 (QUIC), 802.11 WiFi, IPv4/IPv6

Finding the Nearest Server

When you type example.com, DNS does not return one IP. CDNs use GeoDNS or Anycast to direct you to the nearest Point of Presence.

Anycast means hundreds of servers share the same IP. BGP routing directs your packet to the closest server based on network topology. If a PoP fails, traffic reroutes automatically.

Cloudflare: 300+ PoPs | Akamai: 4,000+ edges

HTTP/1 -> HTTP/2 -> HTTP/3

HTTP/1.1 (1997): 6 connection limit, head-of-line blocking

HTTP/2 (2015): Multiplexing over TCP, but still blocked at transport layer

HTTP/3 (RFC 9114): QUIC over UDP, independent streams, 0-RTT resumption

HTTP/3 cuts page load times by 30-50% on mobile/lossy networks

Zero Round-Trip Time

TLS 1.3 (RFC 8446) reduces handshake from 2 RTTs to 1 RTT for new connections, and 0-RTT for returning visitors.

On repeat visits, the client sends encrypted data in the very first packet. No handshake needed. For a cached resource at the edge, this means instant delivery.

TLS 1.2: 2 RTT | TLS 1.3: 1 RTT | TLS 1.3 + 0-RTT: 0 RTT

The Three-Layer Cache Hierarchy

EDGE: Closest to users. Serves 80-95% of requests. Cache TTL: minutes to hours.

REGIONAL: Shared across edge nodes. Handles cache misses. Reduces origin load.

ORIGIN: Your actual server. Only hit on true cache misses. Protected by origin shield.

This tiered design, similar to the DNS hierarchy we analyzed in class, means your origin sees only 5-20% of total traffic.

Cache Headers Control Everything

Cache-Control: max-age - How long the CDN can serve without checking origin. Static assets: 1 year. HTML: 5-60 min.

s-maxage vs max-age: s-maxage controls CDN caching; max-age controls browser caching.

stale-while-revalidate: Serve stale content while fetching fresh in background. Users never wait.

Cache-Control: public, max-age=31536000, s-maxage=86400, stale-while-revalidate=3600

The Hardest Problem in CS

1. Purge API: Call the CDN API to invalidate specific URLs instantly. Cloudflare propagates purges across all PoPs in approximately 30 seconds.
2. Cache Busting: Add content hash to filenames: app.abc123.js. Instant invalidation, no purge needed.
3. Surrogate Keys: Tag content with keys. Purge by key to invalidate thousands of URLs at once. Fastly pioneered this.

The Network is the Computer

Cloudflare operates 300+ Points of Presence across 100+ countries, processing 55 million HTTP requests per second. They handle approximately 25% of all web traffic.

Key Innovations:

Workers: V8 isolates running at every edge location. Deploy JavaScript globally in 30 seconds. No cold starts.

R2 Storage: S3-compatible object storage with zero egress fees.

DDoS Mitigation: Mitigated the largest attack ever: 71 million requests per second in February 2023.

Akamai & Fastly

Akamai: 4,000+ edge locations. Enterprise and media streaming focus. Clients: Apple, Adobe, Airbnb.

Fastly: 100+ locations. Real-time config propagation in 150ms. Clients: GitHub, Stripe, Spotify.

AWS CloudFront: 450+ locations. Deep AWS integration. Clients: Slack, Hulu, NASA.

Each provider serves different use cases: Akamai for media, Fastly for real-time config, CloudFront for AWS integration, Cloudflare for developer experience.

Latency vs Cost vs Complexity

Latency Win: CDNs cut latency from 200ms to 20ms, a 10x improvement. But edge compute adds cost per request.

Cost Trade-off: Origin bandwidth drops 80-95%, but CDN egress fees apply. At 90% cache hit ratio, you save approximately \$80/TB.

Complexity Tax: Cache invalidation, SSL certificate management, and vendor-specific APIs add operational overhead.

Power & Energy: Edge servers consume power 24/7. A global CDN uses megawatts of electricity. However, reduced origin load and shorter transmission distances can lower overall network energy consumption.

Cache Invalidation Hell

Thundering Herd: When cache expires, thousands of requests simultaneously hit the origin. Solution: origin shield + stale-while-revalidate.

SSL Termination: CDNs decrypt TLS at the edge. However, end-to-end encryption to origin is available (Cloudflare Full Strict, CloudFront Origin SSL). Without it, data travels in plaintext inside the CDN backbone.

Vendor Lock-in: Edge compute is vendor-specific. Migrating means rewriting edge logic. Multi-CDN reduces risk.

Code at the Edge

I would implement edge compute for dynamic content. Instead of routing every request to origin, run lightweight logic at the edge.

Cloudflare Workers (V8 isolates) and Fastly Compute (WebAssembly) both enable this architecture.

Traditional: User -> CDN -> Origin -> Database -> Origin -> CDN -> User (200-500ms)

Edge-First: User -> Edge Worker -> D1/KV -> User (5-20ms)

HTTP/3 Adoption is Critical

I would enable HTTP/3 immediately on all CDN properties. As of 2026, approximately 35% of web traffic uses HTTP/3.

The holdouts are enterprise firewalls that block UDP and legacy load balancers.

HTTP/3 shines on mobile networks where packet loss is common. On a 3% packet loss connection, HTTP/3 loads pages 2x faster than HTTP/2.

35% of web on HTTP/3 | 2x faster on lossy networks | 0 RTT with resumption

Smarter Cache Prediction

I would invest in ML-driven prefetching. The next frontier is not just caching, but intelligent content distribution that anticipates demand before it happens.

ML-Driven Prefetch: Use machine learning to predict what users will request next. Prefetch to edge before they ask. Netflix uses this for thumbnails.

Tiered TTL: Hot content gets short TTL for freshness. Cold content gets long TTL for efficiency. Dynamically adjusted.

Real-time Purge: WebSocket-based invalidation. When origin updates, push purge events to all edge nodes instantly.

APA References

Cloudflare. (2024). How does a CDN work? Cloudflare Learning Center. <https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>

Fielding, R., & Reschke, J. (2014). Hypertext Transfer Protocol (HTTP/1.1): Conditional Requests (RFC 7232). IETF.

Kurose, J. F., & Ross, K. W. (2021). Computer networking: A top-down approach (8th ed.). Pearson.

Mozilla. (2024). HTTP caching. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Caching>

Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3 (RFC 8446). IETF.

Iyengar, J., & Thomson, M. (2021). QUIC: A UDP-Based Multiplexed and Secure Transport (RFC 9000). IETF.

Bishop, M. (2022). HTTP/3 (RFC 9114). IETF.

IETF. (2022). Transmission Control Protocol (TCP) (RFC 9293). IETF.

Questions & Answers

Kian Kamali

ENEB352 - Introduction to Networks and Protocols

University of Maryland, Spring 2026